German University in Cairo
Department of Computer Science
Assoc. Prof. Mervat Abu El-Kheir

Architecture of Massively Scalable Applications, Spring 2026
Lab Manual 2 - Git & GitHub

Welcome to the Git Lab! In this lab we will practise version control hands-on using a Spring Boot project as our working example. To follow along you need to have **Git** installed on your machine, run `git -version` to confirm. You will also need a **GitHub** account and the project from Lab 1.

1 Downloading the Starter Project

a) Download the lab 1 solution zip file from CMS

b) Extract the zip file and open it in IntelliJ

2 Initialise the Git repository

a) Initialise git in the directory

```
git init
git status
```

Only source files should appear as untracked. The `target/` and `.idea/` folders should be absent.

b) Stage all files and create the first commit:

```
git add .
git commit -m "Initial commit: end of lab 1 code"
```

c) Create an **empty** repository on GitHub (no README, no licence, no `.gitignore`), then connect and push:

```
git remote add origin https:<Your Repo Link>
git branch -M main
git push -u origin main
```

Visit your GitHub repository page to confirm the project structure is visible under your initial commit message.

3 Feature Branch Development

a) Create a feature branch for the new feature (get user profile):

```
git checkout main
git pull origin main
git checkout -b feat/user-profile[1]
```

[1] You can create the branch from GitHub and checkout locally

b) Create a new controller `/users/me` to return a single user (dummy)

```
git add .
git commit -m "feat: add user profile"[2]
git push[3]
```

c) Switch back to `main` and simulate two commits by a teammate:

```
git checkout main
```

d) Change application port in `resources/application.properties`

```
server.port=8081
```

e) Commit your changes

```
git add .
git commit -m "config: set default server port"
git push[4]
```

f) Visualise the divergence:

```
git log --oneline --graph --all
```

You should see the two branches splitting from a common ancestor commit.

4 Rebasing the Feature Branch onto main

Instead of merging (which adds a merge commit), we use `git rebase` to replay our feature commits on top of the latest `main`, keeping a clean linear history.

a) Switch to the feature branch and rebase:

```
git checkout feat/user-profile
git pull --rebase origin main [5]
```

If successful you will see:

`Successfully rebased and updated refs/heads/feat/user-profile.`

b) If a conflict occurs, Git will pause and tell you which file to fix. Open the file, resolve the conflict markers, then:

```
git add <conflicted-file>
git rebase --continue
```

To cancel the rebase entirely:

```
git rebase --abort
```

[2] Check the link in the end of the slides for the best practices on writing a commit meesage
[3] After running this command, git will prompt you to set the upstream of the branch first!!
[4] Note that this time, it is excuted normally as we already sat the upstream so no need to re-do it
[5] this command is equivalent to ***git rebase origin/main***

c) Verify the history is now linear:

```
git log --oneline --graph --all
```

The new controller commit should appear on top of the `main` commit with no merge commit in between.

d) Force-push the rebased branch. Because rebase rewrites commit hashes, a standard push will be rejected:

```
git push origin feat/user-profile --force-with-lease
```
[6]

if someone else pushed to the branch since your last fetch.

Warning: Never rebase a branch that is actively being used by other developers on a shared remote. Rewriting history forces everyone else to re-sync their work.

5 Accidentally Committing a Secret and Fixing It

Note: The API key used below is a dummy value for demonstration only. Never commit real credentials to any repository.

Make sure you are on the feature branch, then create a properties file that accidentally contains a hardcoded API key:

```
git checkout feat/user-profile
```

Update application.properties with:

```
notification.api.url=https://api.notifications.example.com
notification.api.key=sk-DUMMY1234567890abcdef1234567890ABCDEF
notification.api.timeout=5000
```

and then commit these changes

```
git add .
git commit -m "config: add notification API configuration"
```

Revert the commit. `git revert` creates a new commit that undoes the changes of the target, which is safe for shared branches because it does not alter existing history:

```
git log --oneline       # copy the hash of the bad commit
git revert <hash> --no-edit
```

Even after reverting, the API key still exists inside the old commit object. To permanently erase it from the entire history, we can use interactive rebase or `git filter-branch`

```
git rebase -i <hash>^

#replace pick with drop in the commit tree
```

Note: Here, you will get a warning saying:

```
The previous cherry-pick is now empty, possibly due to conflict resolution.
If you wish to commit it anyway, use:
```

[6] `-force-with-lease` is safer than `-force` - it fails

```
    git commit --allow-empty

Otherwise, please use 'git rebase --skip'
interactive rebase in progress; onto <hash>
Last commands done (2 commands done):
   drop 16ad37f config: add notification API configuration
   pick f3ba71e Revert "config: add notification API configuration"
No commands remaining.
You are currently rebasing branch 'feat/user-profile' on <hash>.
  (all conflicts fixed: run "git rebase --continue")
```

This happens because we already dropped the original commit, the revert commit has nothing to revert anymore - the notification API configuration was never added in the first place in this new history. This makes the revert commit "empty" (no changes). In other words we're essentially trying to:

a) Remove a commit (drop)

b) Then apply a commit that undoes what was removed (revert)

These two actions cancel each other out, making the second operation meaningless. Since you already dropped the original commit, you don't need the revert commit anymore.

What we can do is one of two options:

a) ***git rebase –skip***:
Which will skip the now-redundant revert commit (as we already dropped it), this will result in removing the commit entirely from history, have no empty commit and our git log will be cleaner

b) ***git commit –allow-empty + git rebase –continue***:
Which creates an empty commit with the message "*Revert 'config: add notification API configuration'*" as this commit will exist in the repo history but changes zero files.

After finishing the rebase, run:

```
git push origin --force-with-lease
```

Verify the change is completely gone from history:

```
git log --all --full-history -- src/main/resources/application.properties
```

This command should return no commit related to the edits that were made to the application.properties file as we have removed it.

6 Cherry-Picking Commits

Cherry-picking lets you apply a specific commit from one branch to another without merging the entire branch. This is useful when only part of a feature is ready to ship, or when backporting a fix to a release branch.

a) Create a third branch from `main`:

```
git checkout main
git pull origin main [7]
git checkout -b feat/user-profile-2
```

[7] Always pull before creating a new branch, to make sure it has latest changes from main and reduce the chances of falling into merge conflicts.

b) Inspect the log of the source branch to find the commits you want:

```
git log --oneline feat/user-profile
```

We want to copy the controller commit made on the first branch. The secret-related commits should be excluded.

c) Apply the selected commits one at a time, oldest first:

```
git cherry-pick <hash>
```

Replace the hashes above with your actual values from the log output.

d) If a conflict occurs during cherry-pick:

```
git add <conflicted-file>
git cherry-pick --continue

# To abandon:
git cherry-pick --abort
```

e) Push the branch and view the final state of all branches:

```
git push origin feat/user-profile-2
git log --oneline --graph --all --decorate
```

You should now have three branches: `main`, `feat/user-profile`, and `feat/user-profile-2`. The third branch contains only the cherry-picked commit.